

项目开题报告

从 PA 到自研架构模拟器：RISC-V32 与 LoongArch32 双 路线设计与实现

—— 基于南京大学 PA 项目与 LoongArch32 裸机 CPU 模拟器项目的联合开题方案

项目名称	RISC-V32 与 LoongArch32 双路线模拟器项目
项目性质	课程改革项目 / 体系结构综合实践
技术路线	① 基于南京大学 PA 的 RISC-V32 路线 ② 从零构建 LoongArch32 裸机 CPU 模拟器路线
项目成员	张义搏 王玮明 王明军
报告用途	课程开题与阶段性方案汇报
提交日期	2026 年 3 月

关键词：模拟器；RISC-V32；LoongArch32；PA；裸机 CPU；ISA；AM/NEMU

摘要

本项目面向“教学驱动 + 自主实现”两条技术路线，围绕模拟器这一核心载体，开展 RISC-V32 与 LoongArch32 两种指令集架构的联合研究与实现。其中，一条路线基于南京大学 PA 项目，依托 NEMU/AM 提供的教学框架，系统理解 CPU 执行模型、内存系统、I/O、调试器与中断异常等关键机制，目前已推进至 PA3 中断阶段；另一条路线面向 LoongArch32，自底向上完成裸机 CPU 模拟器设计、程序加载、最小 runtime、交叉编译器接入及程序级自动化测试，已能够批量运行一组代表性 C 程序。本联合开题报告的目标，不是简单并列两个子项目，而是从“抽象—实现—验证—扩展”的统一框架出发，构建一套可解释、可演示、可持续演进的多 ISA 模拟器实践方案。

报告将从项目背景、研究意义、已有基础、总体技术路线、关键模块设计、双路线分工、预期成果与进度安排等方面展开。项目最终希望形成两方面成果：其一，借助 PA 路线梳理教学模拟器的分层设计方法，建立对 CPU、存储器、设备与调试机制的系统理解；其二，在 LoongArch32 路线中完成自主 ISA 的模拟器实现闭环，体现国产架构背景下从零搭建处理器执行环境的能力。两条路线相互支撑：PA 路线提供成熟的框架理解与调试方法，LoongArch32 路线提供从零构建、自主适配与工程扩展能力。

报告内容提要

维度	RISC-V32 (PA 路线)	LoongArch32 (自研路线)	联合价值
研究定位	借助教学框架掌握模拟器核心机制	从零实现裸机 CPU 模拟器	形成“理解成熟框架 + 自主实现”双视角
当前基础	已到 PA3 中断，具备 CPU/存储器/设备/调试器理解	已建立 CPU、加载器、runtime 与测试链路	具备继续扩展 MMU、异常、测试集的条件
后续重点	梳理框架、补足 MMU/分页理解并形成汇报体系	补充指令支持、扩展测试覆盖、完善工具链适配	为多 ISA 教学模拟器/自主架构实践提供经验

一、项目背景与研究意义

1.1 项目背景

在计算机系统课程中，学生往往能够理解指令格式、流水线、存储层次和异常机制等理论概念，但这些概念若缺乏“可运行系统”的承载，往往停留在静态知识层面。模拟器恰好提供了一个中间层：它既不像真实芯片那样实现成本高，也不像纯文档那样抽象，可以把 CPU、内存、设备、程序加载与调试机制统一到一套可执行的软件系统中。

本项目来源于课程改革任务，三人小组采用双路线推进：一条路线依托南京大学 PA 项目完成 RISC-V32 模拟器学习与验证，另一条路线面向 LoongArch32，从零构建裸机 CPU 模拟器与验证环境。前者强调“通过成熟教学框架理解系统设计”，后者强调“面向自主架构完成独立工程实现”。

1.2 研究意义

（1）加深对 ISA 与模拟器分层设计的工程化理解。通过 PA 路线，可以系统学习 NEMU/AM 的分层思想，理解 CPU 执行、地址访问、MMIO、SDB、ITrace、异常与中断等模块如何协同工作。

（2）建立“从源程序到机器执行”的完整视角。LoongArch32 路线不仅关注指令语义，还覆盖了加载器、runtime、链接、交叉编译、测试协议等环节，使项目从“能执行若干手写指令”推进到“能批量执行真实编译器产物”。

（3）体现自主架构背景下的实践价值。LoongArch 作为国产指令集，具备很强的教学与研究价值。通过自建 LoongArch32 裸机 CPU 模拟器，可以更具体地理解自主 ISA 的设计、适配与验证过程。

（4）形成可迁移的方法论。PA 路线提供的是‘先理解成熟系统，再提炼框架’的方法，LoongArch32 路线提供的是‘先搭骨架，再逐步闭环’的方法。二者结合后，对后续其他 ISA 或系统软件项目具有迁移价值。

二、项目基础与问题提出

2.1 RISC-V32（PA 路线）已有基础

目前项目负责人已完成南京大学 PA 路线中的 PA1、PA2 回顾，并推进至 PA3 中断阶段。围绕 NEMU 已完成或系统梳理的核心内容包括：主函数与初始化流程、monitor/sdb、表达式求值与监

视点、CPU 单步执行框架、内存系统与 paddr/vaddr 分层、MMIO 与设备映射、ITrace/IRingBuf 调试机制、内置镜像与外部镜像加载、中断与异常控制流等。

从理解层面看，PA 路线已经建立了较清晰的主线：``init_monitor`` → `init_mem/init_device/init_isa/load_img` → `engine_start` → `cpu_exec` → `execute` → `exec_once` → `isa_exec_once` → `decode_exec``。这为后续汇报中讲解教学型模拟器的系统结构提供了可靠基础。

2.2 LoongArch32（自研路线）已有基础

当前已完成的关键工作包括：LoongArch32 CPU 基础执行框架、寄存器与 PC、内存/总线/设备模型、基于 MMIO 的测试设备、程序加载器、.hex 程序验证、程序级自动化测试框架、最小 C runtime、手动构建 LoongArch32 裸机交叉编译器、以及若干真实 C 程序的编译与运行闭环。项目已支持基础算术、条件分支、普通函数调用、单递归和双递归等程序。

2.3 当前问题与开题聚焦

几个需要系统化推进的问题：

- （1）如何把 PA 路线中的‘教学框架理解’上升为可表达的方法论；
- （2）如何把 LoongArch32 路线中的‘已实现模块’重组为面向后续扩展的正式项目方案；

三、项目目标

3.1 总体目标

围绕模拟器这一统一主题，构建“教学框架理解 + 自主架构实现”的双路线实践体系：一方面完成对 RISC-V32 PA/NEMU/AM 体系的系统梳理与汇报表达；另一方面继续完善 LoongArch32 裸机 CPU 模拟器，实现更强的程序运行能力、更高的测试覆盖和更完整的工程闭环。

3.2 分目标

目标层级	RISC-V32（PA）	LoongArch32	联合产出
基础目标	完成 NEMU 启动、执行、调试、设备、异常主线梳理	保持 CPU/内存/设备/加载器/测试链条稳定	形成统一系统框架图与术语说明

中期目标	补充 MMU/分页、AM 五大模块、教学型模拟器方法论表达	继续扩充指令支持、完善 runtime 与 test_def 接入	形成双路线对照分析与阶段性汇报
扩展目标	沉淀可迁移的模拟器设计经验	向更完整验证平台推进	形成可复用项目文档与演示材料

四、总体技术路线与系统结构

联合项目采用“共性主线 + 双路线分支”的技术路线。共性主线强调模拟器的一般结构：CPU 核心、内存系统、地址访问、设备模型、程序加载、调试支持与测试闭环。在此基础上，RISC-V32 路线以 PA/NEMU 为研究对象提炼方法，LoongArch32 路线以自研裸机 CPU 模拟器为实现对象推进工程。

4.1 共性主线

程序/镜像 → 加载到客户机内存 → 初始化 CPU/PC → 单步执行（取指/译码/执行） → 地址访问（VA/PA/MMIO） → 设备交互 → 调试与测试 → 程序退出判定

4.2 双路线对应关系

共性模块	PA / NEMU 侧理解对象	LoongArch32 自研侧实现对象	拟关注的汇报重点
CPU 执行	cpu_exec / execute / exec_once / decode_exec	step/execute/译码执行框架	一条指令如何执行完
内存访问	vaddr/paddr/guest_to_host/MMIO	Memory/Bus/TestDevice/MMIO	普通内存与 I/O 如何区分
程序启动	init_isa/load_img/RESET_VECTOR	加载器、入口地址、linker.ld	从镜像到第一条指令
调试机制	SDB、expr、watchpoint、itrace、iringbuf	批量测试、日志、PASS/FAIL 协议	如何验证系统正确性
系统控制	异常、中断、AM 抽象	异常入口、测试设备、runtime	程序为何能停下/返回

五、RISC-V32 (PA) 路线设计与计划

5.1 路线定位

PA 路线不以“重写一遍 NEMU”为目标，而以“通过实现与源码梳理理解教学型模拟器的分层结构”为目标。当前阶段的重点是把已经完成和理解到的内容组织成清晰可讲的系统框架。

5.2 当前重点模块

- (1) 监控器与调试器：包括 `sdb\_mainloop`、命令解析、表达式求值、监视点、`x N EXPR` 内存扫描等；
- (2) CPU 执行主线：从 `cpu\_exec` 到 `decode\_exec`，重点讲清 `Decode s`、`pc/snpc/dnpc` 与控制流更新；
- (3) 存储器实现：区分 `pmem`、`io\_space`、`guest\_to\_host`、`paddr\_read/write` 与 `mmio\_read/write`；
- (4) 设备模型：串口、定时器、VGA、键盘等通过 MMIO 注册到物理地址空间；
- (5) 异常与中断：围绕 `ecall`、`mret`、CSR、异常入口与返回机制，形成对 PA3 的系统总结。

5.3 后续补充方向

在报告层面，PA 路线后续重点是：补足 MMU/分页知识的表达，解释 AM 五大模块

(TRM/IOE/CTE/VME/MPE) 的作用，并把 NEMU/AM 的分层思想总结为可迁移的方法论。若时间允许，可继续向 PA4 内容做概念性预研与接口接入。

六、LoongArch32 自研路线设计与计划

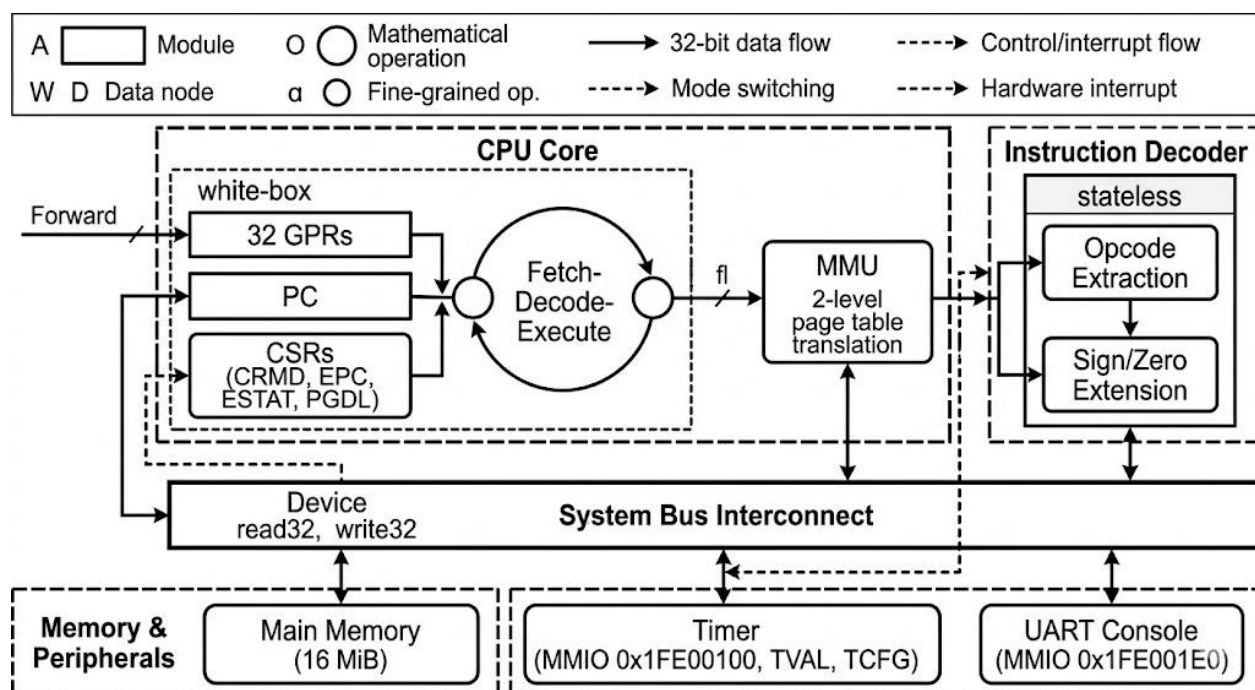


图 1: Loongarch32 虚拟机构建架构图

6.1 当前成果基础

LoongArch32 路线目前已经完成从“手写 .hex 程序测试”到“运行真实 C 程序”的重要跃迁。已建立的关键能力包括:

- (1) CPU 基础骨架: 32 个通用寄存器、PC、CSR 结构、单步执行接口以及架构不变式维护;
- (2) 内存/总线/设备: 主存、总线分发、Timer、UART、TestDevice 等外围组件;
- (3) 程序加载与测试协议: 支持 .hex 加载、裸机 PASS/FAIL 协议、程序级自动化测试;
- (4) 最小 C runtime: 包括启动代码、trap 接口、linker.ld 等;
- (5) 工具链接入: 手动构建 LoongArch32 裸机交叉编译器, 支持 GCC 生成程序的编译与加载。

6.2 后续实施重点

下一阶段将继续围绕“真实编译器产物驱动 CPU 完善”推进: 一方面补齐由编译器实际生成、但当前模拟器尚未覆盖的指令; 另一方面扩大程序验证集合, 逐步接入 test_def 测试集, 提高自动回归验证能力。

6.3 拟重点突破问题

- (1) 继续完善指令支持，覆盖乘除、访存边界、控制流与系统指令；
- (2) 增强异常/中断/MMU 基础框架的一致性；
- (3) 让加载器、runtime、编译器与模拟器在地址空间和调用约定上更加稳固；
- (4) 提升测试框架的可扩展性，使程序清单、预期返回值与日志输出更加规范。

七、两条路线的结合方式与创新点

7.1 结合方式

本项目并非将两条路线简单并列，而是通过“框架理解—自主实现”的对应关系实现结合：PA 路线提供成熟教学框架下的模块分层、调试方法和概念解释；LoongArch32 路线则将这些理念在自主 ISA 环境中重新落地，验证哪些是模拟器设计的共性，哪些是具体 ISA 的特性。

7.2 主要创新点

- (1) 双路线结构创新：把教学型成熟框架与自主架构实现并行推进，形成相互印证的项目模式；
- (2) 方法论创新：从 PA 中提炼抽象层次、从 LoongArch32 中验证工程闭环，最终形成可迁移的模拟器实现框架；
- (3) 国产化价值：在理解 RISC-V32 的同时，进一步落地 LoongArch32 裸机模拟器，体现面向自主 ISA 的工程实践；
- (4) 验证体系创新：强调程序级自动测试、PASS/FAIL 协议、runtime 与工具链联动，而不只停留在单条指令演示。

八、研究方法、实施步骤与进度安排

8.1 研究方法

项目采用“先搭主线、后补细节；先建立闭环、再扩大覆盖；先形成可运行系统、再追求完整性”的方法。具体上体现为：先以最小可运行系统建立执行闭环，再通过测试驱动逐步补齐指令与模块，并通过日志、反汇编、监视点和程序级回归测试定位问题。

8.2 进度安排

阶段	时间安排	主要任务	输出成果
阶段一	已完成	RISC-V32 路线推进至 PA3 中断； LoongArch32 建立 CPU/加载器/runtime/测试链路	阶段性实现基础
阶段二	第 1 - 2 周	整理 PA 路线系统框架，形成汇报图示与概念讲稿；同步梳理 LoongArch32 现有模块	开题汇报材料
阶段三	第 3 - 5 周	LoongArch32 扩充指令支持与测试样例；PA 路线补足 MMU/AM 五大模块表达	中期检查文档
阶段四	第 6 - 8 周	完善程序级回归测试与 test_def 接入，形成双路线对照分析	阶段报告与演示
阶段五	课程末	整合最终总结、实验结果、技术反思与可扩展方向	结题报告/答辩材料

九、预期成果

- （1）形成一份系统完整、逻辑清晰的双路线项目报告与汇报材料；
- （2）完成对 RISC-V32 PA/NEMU/AM 路线的系统梳理，形成可讲解的框架图与关键机制说明；
- （3）在 LoongArch32 路线上继续提升模拟器对真实编译器产物的支持能力，并扩大测试覆盖；
- （4）沉淀一套适用于模拟器项目的工程方法，包括分层设计、最小闭环、程序级验证与持续回归测试；

(5) 为后续继续扩展 MMU、异常、设备与更完整测试集提供文档与代码基础。

十、成员分工

项目采用三人协作方式，其中一人主负责 RISC-V32 PA 路线，两人主负责 LoongArch32 自研路线，同时在测试、方案讨论与汇报材料整合上相互协作。

成员方向	主要职责	协同关系
张义搏	推进 PA 项目、梳理 NEMU/AM 框架、完成 CPU/存储器/设备/调试器/异常主线总结，并负责总汇报的系统框架部分	为 LoongArch32 路线提供教学框架参考与调试思路
王玮明	基于 LoongArch32 虚拟机基础框架与内核设计，包括 CPU 核心、内存/总线/设备及基本执行逻辑的实现与维护	与王明军协作完成 runtime 与工具链接入
王明军	测试验证架构搭建与 CPU 内核完善。测试框架包括加载器、runtime、自动化测试、交叉编译工具链与程序验证	与王玮明协作扩展指令支持并支撑整体验证

十一、风险分析与应对措施

风险点	可能表现	应对策略
指令覆盖不足	编译器生成程序在某些位置触发非法指令或结果错误	通过反汇编 + 最小测试程序 + 回归清单逐步补齐
地址空间与加载不一致	程序能编译但无法正确取指/访存	严格校对 linker.ld、入口地址、加载器与模拟器内存布局
异常/中断路径不稳定	复杂程序或设备交互下出现控制流错误	先建立最小正确路径，再增加状态保存与回归测试
汇报内容碎片化	两条路线难以整合成一个主题	采用统一主线：抽象、执行、

		访存、设备、验证、扩展
--	--	-------------

十二、参考资料

- 南京大学 ProjectN / PA (Programming Assignment) 相关资料与 NEMU 源码。
- Abstract Machine (AM) 与 klib 相关资料。
- LoongArch 体系结构手册及相关公开资料。
- 现有《LoongArch32 裸机 CPU 模拟器项目汇报书》中的阶段性实现内容。
- 课程提供的测试框架、am-kernels 与 cpu-tests 相关程序。

十三、项目亮点

双系统交叉印证，优势互补：项目打破了独立开发的孤岛，不是简单并列两个子项目，而是形成了一种“相互印证”的创新双轨模式。RISC-V32 (PA) 路线提供成熟的系统认知与调试方法，而 LoongArch32 自研路线则负责验证从零构建与工程落地的能力，两条路线在底层逻辑上相互支撑。

构建统一的回归测试与验证体系：在共同测试方面，项目突破了单条指令演示的局限，建立了一套统一的验证体系。它强调程序级的自动化测试、PASS/FAIL 验证协议，以及 runtime 与工具链的联动，用同一套工程化的高标准来检验模拟器的可靠性。

通过对比探索不同架构虚拟机的性能边界：本项目计划通过多个相同测试样例的输入，对比两种基于不同指令集构建的虚拟机的性能，探索计算机体系结构中虚拟机组成、结构对运行程序效率的影响边界。

结语

总体来看，本项目的价值不只在于“写出一个能运行的模拟器”，更在于通过双路线实践，把体系结构课程中的关键概念——指令语义、地址访问、设备抽象、程序加载、异常控制与测试验证——落实为一个真正可执行、可分析、可扩展的系统。RISC-V32 的 PA 路线帮助我们以成熟框架建立系统认知，LoongArch32 的自研路线帮助我们验证自主实现能力。二者结合，将使本项目既具教学深度，也具工程实践意义。